

תכנות מונחה עצמים**עבודת הגשה 3 – הכלה ואופרטורים**

מועד הגשה: 4/5/2026 בשעה 23:50

הוראות הגשה:

1. **אנא קראו בעיון את כל תיאור העבודה בטרם תתחילו לכתוב קוד.**
2. הגשה באופן עצמאי בלבד. הגשה בקבוצות תוביל לציון 0 בעבודה.
3. אין לשתף או להעתיק את העבודה או חלקים ממנה. עבירה על הוראה זו תוביל לציון 0 בעבודה.
4. הגשה דרך מערכת מודול בלבד. שום עבודה לא מתקבלת במייל!
5. **למחלקות המפורטות בעבודה ניתן להוסיף מתודות/אופרטורים נוספות אך אסור להוסיף שדות.**
6. יש למקם כל מחלקה שיהיה עליכם ליצור, בשני קבצים נפרדים H ו-CPP. יש להכניס את החלק התיאורטי בקובץ וורד נפרד. יש להכניס את כל הקבצים של החלק המעשי + קובץ הוורד לתיקיה בשם Ex4, ואז לכווץ יחד. נדרש להגיש קובץ אחד בפורמט RAR או ZIP המכיל את כל הקבצים של כל השאלות. לקובץ המכווץ יהיה שם המהווה את מספר ת.ז. של המגיש.
7. **שאלות ובקשות בקשר לעבודה להפנות אך ורק למרצה האחראית לתרגיל, סבטלנה רוסיין, במייל: sceassign2016@gmail.com.**
8. הקלטים יהיו מהטיפוסים החוקיים.
9. אחרי כל הדפסה יש לבצע ירידת שורה.
10. בתרגיל מותר להשתמש בספריות iostream בלבד!
11. יש להקפיד על תכנות נכון:
 - a. כל הערכים שהם קבועים, (מבחינה לוגית הם לא אמורים להשתנות), חייבים להיות מוגדרים כ: const, define או enum, בהתאם לצורך.
 - b. יש לרשום הערות לכל מחלקה ומתודה מה התפקיד שלה.
 - c. יש להקפיד על כימוס נכון - כל השדות ומתודות השירות ב-private והממשק ב-public! כמו כן: חלוקה לקבצים! כל מחלקה בקבצים נפרדים!
 - d. יש להקפיד על הזחות, כיתוב נכון וקריא, שמות משמעותיים.
 - e. יש לנסות ולייעל את הקוד והתוכנית ככל שניתן. הקפידו על reuse בקוד.
 - f. לפני בקשת קלט (cin) יש להדפיס למשתמש הוראה (cout) איזה קלט מבוקש.

חלק א' – תאורטי (מענה בקובץ טקסט – וורד): 7 נקודות

- (1) מה היא המוטיבציה להעמסת אופרטור כפונקציית friend? יש לתת דוגמה.
- (2) האם ניתן ליצור במחלקה אופרטור מסוג Static? יש לתת הסבר קצר.
- (3) איך ניתן להבדיל בין העמסת אופרטורים postfix ו-prefix?
- (4) יש להשלים את הקוד ולממש בנאי העתקה ע"י שימוש באופרטור השמה. במידה ונדרש יש לעדכן חלקים נוספים באופרטור ההשמה.

```
class Worker{
    int id;
    char* name;
public:
    .....
    Worker (const Worker & );
    Worker & operator=(const Worker & );
};
Worker:: Worker (const Worker & other ){
    //implementation by assignment operator
}
Worker & Worker::operator=(const Worker & other ){
    if(this != & other){
        id=other.id;
        delete[] name;
        name=new char[strlen(other.name)+1];
        strcpy(name,other.name);
    }
    return *this;
}
```

- (5) מה הוא ההבדל בין בנאי העתקה לבין אופרטור השמה (=)?
- (6) האם הטענה הבאה היא טענה נכונה: אופרטור שמחזיר אובייקט מקומי חייב להחזיר אותו *by value*. יש לתת נימוק לתשובה.
- (7) איך ניתן למנוע שימוש באופרטור השמה (=)?

חלק ב' – מעשי (ההגשה היא של קבצי ה- CPP ו-H בלבד) – 93 נקודות:**מערכת "מפענח המילים האבודות"**

תיאור המערכת:

המערכת מנהלת טקסטים עתיקים שהשתבשו במהלך השנים. היא מורכבת מ"מילים עתיקות" המאגדות ל"מגילה" ומופעלת על ידי "מכונת פענוח" המשתמשת במפתח פקודות (מספרים) לתיקון הטקסט.

מבנה המחלקות:

מחלקה AncientWord (המילה) מנהלת:

מערך תווים דינמי - char* data ואורך המילה - int len

יש להגדיר בה את כל הבנאים הדרושים, הורס ואופרטורים:

אופרטור **!** (Case Flip) - הופך אותיות קטנות לגדולות וגדולות לקטנות. האופרטור מחזיר AncientWord&.

אופרטור **~** (Shift Right) מקדם כל תו ב ASCII קדימה ב-1, למשל 'a' הופך ל'b'. יש לשים לב, שהזזה צריכה להיות מעגלית, כלומר 'z' תהפוך ל'a', 'Z' תהפוך ל'A'. מקדם '9' ל'0'. האופרטור מחזיר AncientWord&.

אופרטור **&** (Mirror) הופך את סדר התווים במילה. האופרטור מחזיר AncientWord&.

אופרטור **[]**: גישה לתו ספציפי.אופרטור **<<**: מדפיס את המילה.

למשל, דוגמת דרייבר לבדיקה:

```
int main()
{
    AncientWord word("Abz9");
    cout << word << endl;    // Abz9
    cout << !word << endl;    // aBZ9
    cout << ~word << endl;    // bCA0
    cout << &word << endl;    // 0ACb
    cout << word[1] << endl;    // A

    return 0;
}
```

מחלקה AncientScroll (מגילה - מערך דינאמי של מילים) מנהלת:

מערך דינמי בשם words של מצביעים לאובייקטים AncientWord (AncientWord**), כולל כמות המילים count.

יש להגדיר בה את כל הבנאים הדרושים, הורס ואופרטורים:

אופרטור +=: הוספת מילה חדשה לסוף המגילה.

אופרטור & : הופך את סדר המילים במגילה (המילה האחרונה הופכת לראשונה).

אופרטור []: מחזיר רפרנס למילה ה-i-מאפשר גישה כפולה scroll[i][j]

אופרטור <<: מדפיס את המגילה עם סימני "-" בין המילים.

למשל , דוגמת דרייבר לבדיקה :

```
int main() {
    AncientScroll myScroll;
    myScroll += "Abz9";
    myScroll += "Test";
}
```

ראשוני מצב הדפסת //

```
cout << "0. Start:  " << myScroll << endl;
```

```
// פלט: Abz9-Test
```

!אחת בשורה המעודכנת התוצאה את מדפיס וגם האובייקט את מעדכן גם 1. //

```
cout << "1. After ! : " << (!myScroll[0]) << endl;
```

// פלט aBZ9 (ל-a, ל-z, ל-A הפכה ה) (ל-Z)

A-ל שהופך Z-וה 0-ל שהופך 9-ה של הקסם קורה כאן 2. //

```
cout << "2. After ~ : " << (~myScroll[0]) << endl;
```

(מעגלית 1-ב קודם הכל) bCA0 :פלט //

ומדפיס המילה את הופך. 3 //

```
cout << "3. After & : " << (&myScroll[0]) << endl;
```

```
// טל: 0ACb
```

ומדפיס המגילה כל את הופך. 4. //

```
cout << "4. Scroll &: " << (&myScroll) << endl;
```

```
// טל: Test-0ACb
```

```
cout<<myScroll[0][1];
```

//e

```
return 0;
```

}

מחלקה DecoderMachine (המכונה והמפתח):

זה ה"מוח" המערכת. היא מקבלת "מפתח" ומצבעת פקודות על המגילה.

המחלקה מנהלת מצביע ל AncientScroll ומערך דינמי של מספרים (פקודות) (int* key) , כולל גודל של המערך keyLen.

המפתח (השלישיות): [קוד פעולה], [אינדקס מילה], [פרמטר].

קודי פעולה: 1- הפעלת אופרטור ! (Flip), 2- הפעלת אופרטור ~ (Shift), 3- הפעלת אופרטור & (Mirror).

אינדקס מילה: 0,1,2... הפעלת הפקודה על מילה ספציפית במגילה.

פרמטר : כרגע ערך 0 (שמור להרחבה עתידית).

יש להגדיר בה את כל הבנאים הדרושים, הורס ואופרטורים:

אופרטור **+=**: טעינת מפתח חדש, כלומר ההוספה (מסתיים ב-2-).
אופרטור **!** (Run): ביצוע כל הפקודות שבמפתח על המגילה.
דרישה למימוש אופרטור **!** (Run) במחלקת DecoderMachine:
האופרטור אחראי על הוצאה לפועל של כל פקודות הפענוח הצבורות במפתח (key) על גבי המגילה.
דגשי מימוש:
ביצוע סדרתי: יש לסרוק את מערך המפתח בשלושת (Action, Index, Parameter) ולהפעיל את האופרטור המתאים על המילה הרלוונטית במגילה.
עקרון ה"ביצוע החד-פעמי" (Flush Logic): פקודות במכונת פענוח הן פקודות לביצוע, לא תכונות קבועות. ברגע שפקודה "רצה" (Executed), השפעתה כבר הוחלה על הטקסט במגילה. לכן, על האופרטור לדאוג לניקוי מערך המפתח (key) ואיפוס גודלו בתום הביצוע.
רמז: מניעת כפל פענוח (Double Processing) היא קריטית כאשר מעתיקים מכונה אחת לשנייה או כאשר מריצים את המכונה מספר פעמים.
ערך חזרתי: האופרטור יאפשר שרשור פקודות. למשל: (machine += newData)!, ולכן עליו להחזיר רפרנס לאובייקט הנוכחי.

להלן הדרייבר (פונקציית main) שמדגים צעד אחרי צעד איך המכונה מקבלת מגילה, טוענת מפתח פקודות ומבצעת את הפענוח על כל המילים במקביל.
גמישות המפתח: המערך keyData יכול להיות ארוך מאוד. המכונה תמשיך לקרוא שלשות עד שהיא תקבל ערך 2-.

למשל, דוגמת דרייבר לבדיקה:

```
#include "DecoderMachine.h"

using namespace std;

int main() {
    // --- בדיקת 1: חלק ---
    cout << "--- Testing AncientWord ---" << endl;
    AncientWord word("Abz9");
    cout << "Original: " << word << endl;           // Abz9
    cout << "After ! : " << !word << endl;           // aBZ9 (Case Flip)
    cout << "After ~ : " << ~word << endl;           // bCA0 (Shift)
    cout << "After & : " << &word << endl;           // 0ACb (Mirror)
    cout << "Index 1: " << word[1] << endl;         // A
    cout << endl;

    // --- בדיקת 2: חלק ---
    cout << "--- Testing AncientScroll ---" << endl;
    AncientScroll myScroll;
```

```

myScroll += "Abz9";
myScroll += "Test";

cout << "0. Start:  " << myScroll << endl; // Abz9-Test

// מילה שינוי: כפולה וגישה שרשור בדיקת
cout << "1. After ! : " << !myScroll[0] << endl; // aBZ9

cout << "2. After ~ : " << ~myScroll[0] << endl; // bCA0

cout << "3. After & : " << &myScroll[0] << endl; // 0ACb

// כולה המגילה סדר היפוך
cout << "4. Scroll &: " << &myScroll << endl; // Test-0ACb

// scroll[i][j] כפולה גישה בדיקת
cout << "Access [0][1]: " << myScroll[0][1] << endl; // e (מתוך Test)
cout << endl;

// --- המכונה DecoderMachine בדיקת: 3 חלק ---
cout << "--- Testing DecoderMachine ---" << endl;

//Test-0ACb - scroll
// myScroll של דינמי עותק לעצמה בונה היא - מכונה יצירת
DecoderMachine machine(myScroll);
cout<<machine<<endl;//Test-0ACb

// [קוד, של שלשות: מפתח הגדרת]
// 2- ב- מסתיים. 1=Flip, 2=Shift, 3=Mirror.
int keyData[] = {
    1, 1, 0, // 0ACb) 1 באינדקס מילה על הפעל Flip
    2, 1, 0, // 1 באינדקס מילה על הפעל Shift
    3, 0, 0, // 0 באינדקס מילה על הפעל Mirror
    -2 // מפתח סוף
};

// (אופרטורים שרשור) אחת בשורה והרצה טעינה
!(machine += keyData);

cout << "Final Results:" << endl;
cout << "Machine Decoded Scroll: " << machine << endl;
cout << "Original Scroll (Unchanged): " << myScroll << endl;

// --- השמה ואופרטור מעתיק בנאי בדיקת: 4 חלק ---
cout << "\n--- Testing Copy & Assignment ---" << endl;
DecoderMachine machine2 = machine; // מעתיק בנאי

```

```
int keyData2[] = {2, 0, 0, -2}; // למילה Shift פקודת עוד
!(machine2 += keyData2);

cout << "Machine 2 (copied & updated): " << machine2 << endl;
cout << "Machine 1 (remains same): " << machine << endl;

return 0;
```

הפלט :

```
--- Testing AncientWord ---
Original: Abz9
After ! : aBZ9
After ~ : bCA0
After & : 0ACb
Index 1: A
```

```
--- Testing AncientScroll ---
0. Start: Abz9-Test
1. After ! : aBZ9
2. After ~ : bCA0
3. After & : 0ACb
4. Scroll &: Test-0ACb
Access [0][1]: e
```

```
--- Testing DecoderMachine ---
Test-0ACb
Final Results:
Machine Decoded Scroll: tseT-1bdC
Original Scroll (Unchanged): Test-0ACb
```

```
--- Testing Copy & Assignment ---
Machine 2 (copied & updated): utfU-1bdC
Machine 1 (remains same): tseT-1bdC
```

עבודה פוריה !!!